TASK 3

CASE STUDY-BASED ANALYSIS AND DESIGN USING PDA AND CFG

A study of balanced parentheses, arithmetic-expression validation and palindrome recognition

---

INDIVIDUAL CASE STUDY REPORT

Submitted by

Syed Abrar C

Register Number: _____

In partial fulfilment of the requirements for the course

FORMAL LANGUAGE AND AUTOMATA (21CSC301T)

BACHELOR OF TECHNOLOGY
COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM
BENGALURU CAMPUS
SCHOOL OF COMPUTING

November 2026

Contents

Section	Title
1	Abstract and Introduction
2	Theoretical Background: CFG and PDA
3	Case Study 1 – Balanced Parentheses
4	Case Study 2 – Arithmetic Expression Validation
5	Case Study 3 – Palindrome Checking
6	Comparative Analysis and Why Finite Automata Are Insufficient
7	Overall Limitations and Future Scope
8	Oral Explanation / Demonstration Plan
9	Conclusion
10	References

Report organization. The report first introduces context-free grammars and pushdown automata, then develops three language-recognition case studies. Each case study defines its alphabet and language, gives a CFG and PDA model, demonstrates a derivation or stack operation, and evaluates accepted and rejected examples. The final sections compare the models, explain the need for stack memory, and identify assumptions and limitations.

1. Abstract and Introduction

1.1 Abstract

This report examines three language-recognition problems using context-free grammars (CFGs) and pushdown automata (PDAs): balanced-parenthesis recognition, arithmetic-expression validation, and palindrome checking over a two-symbol alphabet. The examples illustrate different uses of stack memory. Balanced parentheses require the model to remember how many opening brackets remain unmatched and to respect their nesting order. Arithmetic expressions require the model to preserve pending grammar symbols and nested expression structure while checking token order and operator placement. Palindrome recognition requires the model to store an initial portion of the input and compare it with the remaining portion after the midpoint.

For each case study, the report defines the target language, presents grammar productions, describes a PDA construction, and demonstrates how valid and invalid strings are handled. The report also discusses ambiguity and determinism, and explains why a finite automaton without a stack cannot, in general, recognize these context-free patterns. The models are theoretical recognizers rather than complete software products; their assumptions and limits are stated explicitly.

1.2 Introduction

Formal language theory provides mathematical tools for describing which sequences of symbols belong to a language. These tools are important in compiler construction, syntax checking, protocol analysis, structured input validation and the study of computation. A language can be described by rules that generate valid strings or by a machine that recognizes them. A context-free grammar describes how larger structures are built from smaller structures, while a pushdown automaton recognizes languages using finite control together with a stack.

A finite automaton has only a finite number of states. It can remember a limited amount of information about the input, but it cannot store an arbitrarily large count or an arbitrarily long prefix. A PDA adds a stack, which can grow as the input grows. The stack follows a last-in, first-out discipline: the most recently pushed symbol is the first one removed. This makes it useful for nested structures and for comparing two parts of a string in a controlled way.

The three selected examples provide complementary demonstrations. Balanced parentheses use deterministic push/pop behaviour. Arithmetic-expression validation uses a grammar-simulation PDA, where grammar productions are selected and expected terminals are matched against the input. Palindrome checking uses nondeterminism to guess where the first half ends, then compares the second half against the stack.

1.3 Problem Statement and Objectives

The problem addressed is to construct clear CFG and PDA models for three representative context-free language problems and to demonstrate their behaviour on both valid and invalid inputs. The objective is not to implement a complete compiler or production parser, but to make the formal language, grammar, stack operations and acceptance conditions explicit.

- Define the alphabet and accepted language for each case study.
- Write CFG productions using a clearly identified start variable, non-terminals and terminals.
- Specify the PDA states, input symbols, stack symbols, initial configuration and acceptance condition.
- Show at least one valid derivation or a step-by-step stack trace for each language.
- Test accepted and rejected strings and explain the reason for each result.
- Discuss grammar ambiguity, deterministic or nondeterministic behaviour, assumptions and limitations.
- Compare the role of stack memory with the restricted memory of finite automata.

1.4 Scope and Assumptions

The report follows the three recommended case studies in the supplied Task 3 guide. For balanced parentheses, the alphabet contains only '(' and ')', and the empty string is included. For arithmetic expressions, the input is assumed to be tokenized before parsing: id and num each represent one operand token, and whitespace between tokens is ignored. The operators are +, -, * and /, with multiplication and division assigned higher precedence than addition and subtraction. For palindrome checking, the alphabet is restricted to {a,b}, and both even-length and odd-length palindromes are included.

All examples use standard CFG and PDA concepts. Where the guide presents a conceptual PDA rather than a fully enumerated transition table, this report clarifies the operational interpretation without changing the intended language. A grammar-based PDA may use nondeterministic production choices; the palindrome PDA is explicitly nondeterministic because it must guess the midpoint.

2. Theoretical Background: CFG and PDA

2.1 Context-Free Grammar

A context-free grammar is a formal system used to generate strings in a language. It is commonly represented as G = (V, Σ, P, S), where V is a finite set of variables or non-terminal symbols, Σ is a finite set of terminals, P is a set of production rules, and S is the start variable. A production replaces one non-terminal with a sequence of terminals and/or non-terminals. Repeated application of productions derives strings in the language.

A derivation begins with the start variable and applies productions until only terminal symbols remain. The notation S ⇒ α indicates one derivation step, while S ⇒* w indicates that the terminal string w can be derived in zero or more steps. The empty string is represented by ε. A grammar is useful because it describes the recursive structure of a language rather than listing all valid strings individually.

2.2 Pushdown Automaton

A pushdown automaton is a finite-state machine equipped with a stack. A common formal representation is M = (Q, Σ, Γ, δ, q0, Z0, F), where Q is the set of states, Σ is the input alphabet, Γ is the stack alphabet, δ is the transition relation, q0 is the start state, Z0 is the initial bottom-of-stack symbol, and F is the set of accepting states. Depending on the chosen convention, acceptance may be defined by final state or by empty stack; this report states the acceptance condition used for each design.

A transition can depend on the current state, next input symbol (or ε), and top stack symbol. It may change the state, consume an input symbol, and replace the top stack symbol with a sequence of stack symbols. A push adds information to the stack; a pop removes the top symbol. An ε-transition changes the state or stack without consuming an input token.

2.3 CFG–PDA Relationship

Context-free grammars and nondeterministic pushdown automata describe the same class of languages: the context-free languages. A PDA can simulate a grammar by placing the start variable on its stack, nondeterministically replacing a non-terminal with the right-hand side of a production, and matching terminal symbols on the stack with the next input symbol. It accepts when the input has been consumed and the pending stack content has been cleared, subject to the selected bottom-marker convention.

2.4 Acceptance and Rejection

A string is accepted only when the complete input satisfies the model. Reaching an accepting state before the input ends is not sufficient if extra input remains. Likewise, a stack that becomes empty too early or retains unmatched symbols after the input ends indicates rejection. Rejected examples are useful because they reveal which rule has been violated: unmatched brackets, a missing operand, an incorrect operator sequence, or a mismatch between the two halves of a palindrome.

2.5 Ambiguity and Determinism

A CFG is ambiguous if at least one string in its language has two distinct parse trees (equivalently, two distinct leftmost derivations). Determinism concerns the recognizer: a deterministic PDA has at most one possible move for a given configuration and next input symbol, while a nondeterministic PDA may have several possible moves. These are related but different properties. An unambiguous grammar does not automatically imply that every PDA implementation is deterministic.

3. Case Study 1 – Balanced Parentheses

3.1 Objective and Practical Context

The objective is to determine whether every opening parenthesis '(' has a corresponding closing parenthesis ')' in the correct order. This pattern occurs in mathematical notation, function calls, nested expressions and the structural syntax of programming languages. A valid string may contain several pairs, nested pairs, or consecutive balanced groups. The essential requirement is that a closing parenthesis must never appear when there is no unmatched opening parenthesis available, and no opening parenthesis may remain unmatched when the input ends.

3.2 Alphabet and Language Definition

Σ = { '(', ')' }
L■ = { w ∈ Σ* | w is properly balanced }

The empty string ε belongs to the language because it contains no unmatched parentheses. A useful operational condition is that every prefix of a valid string has at least as many opening parentheses as closing parentheses, and the complete string has equal numbers of each. This prefix condition prevents a closing parenthesis from appearing too early.

3.3 Context-Free Grammar

S → (S)S | ε

S is the start variable. The production S → (S) introduces one matched outer pair around a balanced substring. The final S permits another balanced sequence after that pair. The production S → ε ends the derivation and allows the innermost region or the remaining sequence to be empty. Together, these productions generate every properly balanced one-type parenthesis string.

3.4 Example CFG Derivation for (()())

S ⇒ (S)S
 ⇒ ((S)S)S
 ⇒ (()S)S
 ⇒ (()(S)S)S
 ⇒ (()())S
 ⇒ (()())

The first production creates an outer pair. The inner S is expanded to create the nested pair and the next pair inside it. Each remaining S is eventually replaced by ε. The resulting terminal string is (()()), which is balanced.

3.5 PDA Design and Formal Behaviour

The PDA uses its stack to record opening parentheses that have not yet been matched. It begins with a bottom marker Z0. Whenever it reads '(', it pushes X. Whenever it reads ')', it must find X at the top and pop it. If it reads ')' while Z0 is on top, the input is rejected because no unmatched opening parenthesis exists. At the end of the input, acceptance occurs only when Z0 is the sole stack symbol.

States: Q = {q, qf, qrej} (qrej may be implicit)
Input alphabet: Σ = {'(', ')'}
Stack alphabet: Γ = {Z0, X}
Start state: q
Initial stack symbol: Z0
Accepting state: qf
Acceptance: all input consumed and stack contains only Z0.

Operational transitions (described using the stack top):

Current input	Top of stack	Operation
'('	Z0 or X	Push X above the current stack top; remain in q.
')'	X	Pop X; remain in q.
')'	Z0	No valid move; reject.
End of input	Z0	Move to qf and accept.
End of input	X	Unmatched opening parenthesis remains; reject.

This is deterministic for the one-type parenthesis language: the next input symbol determines whether to push or pop. The model does not need to guess where the input ends or where a matching pair occurs.

3.6 Stack Trace for (()())

The following trace shows the stack after each symbol has been processed. The stack is written bottom-to-top, with the rightmost symbol at the top.

Step	Input read	Stack (bottom → top)	Explanation
0	Start	Z0	Only the bottom marker is present.
1	(	Z0 X	Push X for the first opening parenthesis.
2	(	Z0 X X	Push another X for the nested opening parenthesis.
3	)	Z0 X	Pop one X; the most recent opening is matched.
4	(	Z0 X X	Push X for the next opening parenthesis.
5	)	Z0 X	Pop one X.
6	)	Z0	Pop the remaining X; all openings are matched.
7	End	Z0	Input is consumed and only Z0 remains: accept.

3.7 Test Cases

Accepted string	Reason
()	One opening and one closing parenthesis match.
(())	The inner pair closes before the outer pair.
()()	Two balanced groups appear consecutively.
(()())	All nested and adjacent pairs are matched in order.
ε	The empty string contains no unmatched parentheses.
Rejected string	**Reason**
(	An opening parenthesis remains unmatched at the end.
)	A closing parenthesis appears when the stack contains only Z0.
(()	One opening parenthesis remains unmatched.
())	The third symbol is an extra closing parenthesis.
())(	A closing parenthesis occurs with no matching opening; later input cannot repair that prefix error.

3.8 Ambiguity and Determinism

The grammar $S \rightarrow (S)S \mid \varepsilon$ is an unambiguous grammar for the language of properly balanced strings with one parenthesis type. Its recursive structure identifies an outer matched pair and the balanced sequence that follows it. The stack-based PDA can recognize the same language deterministically because each input symbol has one prescribed action: push for '(' and pop for ')'. The machine rejects immediately if a pop is requested when no X is available, and it rejects at end-of-input if any X remains.

3.9 Limitations

This model recognizes only one kind of bracket, namely round parentheses. It does not directly distinguish square brackets, curly braces, angle brackets or language-specific delimiters. Supporting multiple bracket types would require a larger stack alphabet so that the PDA can remember which opening type must be matched by each closing type. The model also checks only the ordering and balance of delimiters; it does not determine whether the content inside the parentheses is a valid expression, function call, statement or program.

The recognizer assumes that the input consists exclusively of the two defined symbols. If letters, digits, commas, quotation marks or whitespace are present, the alphabet and transition rules must be extended or a preprocessing stage must remove irrelevant characters. Such preprocessing must be designed carefully: removing symbols without understanding quoted strings or comments could cause parentheses that appear inside text literals to be interpreted incorrectly.

The model does not perform semantic validation. For example, it can accept a balanced string such as (()()) without knowing whether the contents represent meaningful mathematical or programming-language constructs. It also does not check whether a function name exists, whether an argument count is correct, or whether an expression is type-compatible. These checks belong to later parsing or semantic-analysis stages.

Finally, the model treats the input as a finite string and assumes that the complete string is available to the recognizer. In a streaming environment, the system may need to maintain the stack across chunks, handle interrupted input, and impose resource limits to prevent extremely deep nesting from consuming excessive memory. The PDA provides the correct theoretical recognition model, but a real implementation still needs error reporting, input-size policies and integration with the surrounding application.

4. Case Study 2 – Arithmetic Expression Validation

4.1 Objective and Practical Context

The objective is to determine whether an arithmetic expression follows a simplified grammar containing operands, binary arithmetic operators and parentheses. This type of syntax validation is a basic part of calculators, interpreters and compilers. The model checks whether operands and operators appear in a structurally valid order and whether parentheses are properly placed. It also demonstrates how a grammar can encode operator precedence.

4.2 Alphabet, Tokens and Assumptions

```
Terminals: { id, num, +, -, *, /, (, ) }<br/>id = identifier token<br/>num = numeric-literal token
```

The terminals id and num represent complete tokens, not individual characters. Tokenization is assumed to have already separated identifiers and numbers from operators and parentheses. Whitespace between tokens is ignored. The grammar treats +, -, * and / as binary operators. Unary minus, decimal-format rules, exponentiation, function calls and implicit multiplication are outside the current language.

4.3 Context-Free Grammar

```
E → E + T | E - T | T<br/>T → T * F | T / F | F<br/>F → ( E ) | id | num
```

E represents an expression, T represents a term, and F represents a factor. The E productions handle addition and subtraction. The T productions handle multiplication and division. Since an expression is built from terms, and terms are built from factors, multiplication and division bind more tightly than addition and subtraction. Parentheses allow a complete expression to be treated as a factor.

The grammar is left-recursive. Left recursion is permitted in a context-free grammar and expresses left associativity for the binary operators at each level. A parser implementation based on recursive descent may need an equivalent transformed grammar, but that implementation concern does not invalidate the CFG.

4.4 Example Derivation for (a+b)*c

```
E ⇒ T<br/> ⇒ T * F<br/> ⇒ F * F<br/> ⇒ (E) * F<br/> ⇒ (E + T) * F<br/> ⇒ (T + T) * F<br/> ⇒ (F + F) * F<br/> ⇒ (a + b) * c
```

The outer expression is first reduced to a term because the top-level operator is multiplication. The term is expanded as T * F. The left factor becomes a parenthesized expression, which is expanded to a+b. The right factor becomes c. This derivation demonstrates that the parentheses group a+b and that the multiplication applies to the grouped result.

4.5 PDA Design: CFG Simulation

A PDA can recognize this grammar by storing pending grammar symbols on its stack. Initially, the stack contains the start variable E above a bottom marker. When a non-terminal is on top of the stack, the PDA may choose one of its grammar productions and replace that non-terminal with the production's right-hand side. When a terminal is on top, the PDA compares it with the next input token. If they match, the terminal is removed from the stack and the input token is consumed. If they do not match, that computation path fails.

```
States: q0 (initialise), qparse (expand variables), qmatch (match terminals), qf (accept)<br/>Input alphabet: tokens id, num, +, -, *, /, (, )<br/>Stack alphabet: grammar variables, terminals, and bottom marker Z0<br/>Initial stack: Z0 E (top convention: E is the
```

```
next symbol to process)<br/>Acceptance: all input tokens consumed and only Z0 remains.
```

Production-expansion and terminal-matching behaviour

Stack top	Input condition	PDA action
E	Any next token	Choose E → E+T, E → E-T, or E → T; replace E by the chosen right-hand side.
T	Any next token	Choose T → T*F, T → T/F, or T → F.
F	Next token is '('	Use F → (E).
F	Next token is id	Use F → id.
F	Next token is num	Use F → num.
Terminal t	Next input token is t	Pop t and consume the input token.
Terminal t	Next input token differs	This computation path fails.

For a conventional stack whose top is at the right, a production right-hand side is pushed in reverse order so that its leftmost symbol is processed first. The grammar-simulation PDA is nondeterministic because it may have to choose among multiple productions for a non-terminal. A successful computation path is sufficient for acceptance.

4.6 Illustrative Stack / Derivation Explanation

For (a+b)*c, the PDA begins with E on the stack. It chooses E → T, then T → T*F. The left T is expanded to F, and that F is expanded to (E). The inner E is expanded to E+T, then each side is reduced through T and F until the operands a and b are matched. The final F on the outer right becomes c. At every stage, the stack stores the symbols that are still expected. The input is accepted when all tokens have been matched and no pending grammar symbols remain.

Stage	Grammar / stack intention	Input portion matched
1	E → T	None
2	T → T*F	None
3	Left T → F → (E)	Opening parenthesis '('
4	Inner E → E+T	Inner operator '+'
5	Left E and right T reduce to operands	a and b
6	Outer right F → id	Closing parenthesis ')' and c are matched in sequence with the remaining stack expectations
7	Stack cleared; input consumed	Accept

4.7 Test Cases

Accepted expression	Reason
a+b	Two operands are joined by a binary addition operator.
a*b+c	A multiplication term is followed by addition.
(a+b)*c	A parenthesized expression is multiplied by c.
num/id	A numeric token is divided by an identifier token.
a-(b/c)	Subtraction uses a parenthesized division expression.
Rejected expression	**Reason**
+a	The expression begins with an operator and has no left operand.

Rejected expression	Reason
a+	The addition operator has no right operand.
(a+b	The opening parenthesis is not closed.
a**b	Two multiplication operators occur consecutively; an operand is missing between them.
(a+)	The right operand after the plus operator is missing.

4.8 Ambiguity and Determinism

The displayed grammar separates expressions, terms and factors. This structure encodes the intended precedence of multiplication and division over addition and subtraction, avoiding the common ambiguity that would arise from a single flat rule such as E → E+E | E*E | id. The grammar is designed to provide a consistent structural interpretation for the supported operators. The left-recursive form also groups repeated operators at a level from the left.

The general CFG-simulation PDA may make nondeterministic production choices. In particular, it can choose a production for E or T before all later input has been examined. Some choices will fail to match the input, while another branch may successfully consume the complete expression. A deterministic parser can be built for suitable transformed grammars and parsing strategies, but the PDA construction described here is the general nondeterministic simulation.

4.9 Limitations

The grammar validates a simplified syntax only. It does not verify whether an identifier has been declared, whether a variable is in scope, whether an operand has a compatible data type, or whether a function or constant exists. The expression a+b can be syntactically valid even if a and b are undefined in the surrounding program. These questions require symbol-table lookup and semantic analysis beyond the CFG presented here.

The model also does not evaluate the expression. It cannot determine the numerical result, detect an arithmetic overflow, or decide whether a division operation will produce an error at runtime. In particular, a syntactically valid expression such as a/0 may still be unsafe or undefined depending on the language's numeric rules. Division-by-zero checking requires value analysis or runtime evaluation rather than grammar recognition.

The tokenization assumption is important. The grammar receives id and num as already identified tokens, so it does not define what counts as a legal identifier or numeric literal. It does not specify whether identifiers may contain underscores, whether numbers may contain decimal points, or whether scientific notation is supported. A lexer must define these rules and produce the token stream that the PDA checks. If tokenization is incorrect, the parser may receive an incorrect sequence even when the original character string appears reasonable.

Only four binary operators are included. Unary operators such as -a, exponentiation, modulo, assignment, function calls, array indexing and implicit multiplication are not represented. Adding these constructs requires new terminals and carefully designed productions to express their precedence and associativity. The current grammar also does not include error-recovery rules. A practical parser should report the location of a missing operand or unmatched parenthesis and may attempt to continue parsing to identify further errors.

Finally, a grammar-based PDA is a formal recognition model, not a complete calculator or compiler. It assumes a finite input token sequence and sufficient stack capacity. A real implementation must address malformed token streams, resource limits, useful diagnostics, source positions and integration with later semantic checks. The report therefore treats acceptance as evidence of syntactic conformity to this grammar only, not proof that an expression is meaningful or executable.

5. Case Study 3 – Palindrome Checking

5.1 Objective and Practical Context

The objective is to recognize strings that read identically from left to right and from right to left. Such strings are called palindromes. Examples include ε, a, b, aa, aba, abba and baab. Palindrome recognition is a useful theoretical example because the recognizer must compare symbols at corresponding positions on opposite sides of the midpoint.

5.2 Alphabet and Language Definition

```
Σ = {a, b}<br/>L■ = { w ∈ {a,b}* | w = w■ }
```

Here w■ denotes the reverse of w. The language includes the empty string, all one-symbol strings, and longer strings whose symbols are symmetric around a central point. Both even-length and odd-length palindromes are included. The alphabet is deliberately restricted to two symbols to keep the grammar and stack behaviour clear.

5.3 Context-Free Grammar

```
S → aSa | bSb | a | b | ε
```

S is the start variable. The recursive productions add the same symbol to the left and right ends of the string. The productions S → a and S → b provide one-symbol centres for odd-length palindromes, while S → ε provides an empty centre for even-length palindromes. Repeated use of the recursive rules therefore preserves the property that the string reads the same in both directions.

5.4 Example Derivation for abba

```
S ⇒ aSa<br/> ⇒ abSba<br/> ⇒ abεba<br/> ⇒ abba
```

The first step places a at both ends. The next step places b at both ends of the remaining centre. Replacing S with ε ends the derivation, producing abba. The outermost and innermost pairs are symmetric, so the resulting string is a palindrome.

5.5 PDA Design: Nondeterministic Midpoint Guess

A nondeterministic PDA recognizes this language by pushing the first part of the input onto its stack. At some point, it guesses that the midpoint has been reached and changes from the push phase to the matching phase. During the matching phase, each remaining input symbol must equal the symbol at the top of the stack; the PDA then pops that stack symbol. If the input length is odd, the PDA consumes one middle symbol when it makes the midpoint guess. It accepts when the entire input is consumed and only the bottom marker remains.

```
States: qstart, qpush, qmatch, qf<br/>Input alphabet: Σ = {a,b}<br/>Stack alphabet: Γ = {Z0,
a, b}<br/>Initial stack symbol: Z0<br/>Acceptance: all input consumed and stack returned to
Z0; final state qf.
```

The PDA has nondeterministic ε-transitions from the push phase to the matching phase. One branch guesses an even-length midpoint without consuming an input symbol. Another branch guesses an odd-length midpoint by consuming exactly one symbol as the centre without pushing it. The remaining suffix is then compared against the stack in reverse order.

5.6 Example Processing of abba

Step	Input action	Stack (bottom → top)	Explanation
0	Start	Z0	Initial bottom marker.
1	Read a; push a	Z0 a	Store the first symbol.
2	Read b; push b	Z0 a b	Store the second symbol.
3	Guess even midpoint	Z0 a b	Switch to matching phase without consuming input.
4	Read b; match and pop	Z0 a	The input b matches the stack top b.
5	Read a; match and pop	Z0	The input a matches the stack top a.
6	End of input	Z0	Input consumed and stack restored to bottom marker: accept.

For an odd-length example such as aba, one successful branch pushes the initial a, guesses the middle when it reads b and consumes that middle symbol without pushing it, then matches the final a against the stack. The input is accepted when the stack returns to Z0.

5.7 Test Cases

Accepted string	Reason
ε	The empty string is equal to its reverse.
a	A one-symbol string is a palindrome.
abba	The reverse of abba is abba.
aba	The reverse of aba is aba.
baab	The reverse of baab is baab.
Rejected string	**Reason**
ab	Its reverse is ba, which is different.
aab	Its reverse is baa.
abb	Its reverse is bba.
abab	Its reverse is baba.
aabb	Its reverse is bbaa.

5.8 Ambiguity and Determinism

The CFG generates palindromes over {a,b} by choosing matching outer symbols recursively. The recursive productions preserve symmetry, and the centre productions determine whether the palindrome has odd or even length. The grammar is structured so that the outermost symbols and the remaining centre determine the derivation for a given string.

A PDA for this language generally requires nondeterminism because it is not told where the midpoint lies. While reading the input from left to right, it cannot know whether another symbol belongs to the first half or whether the matching phase should begin. It therefore guesses the midpoint. A standard deterministic one-stack PDA cannot recognize the full palindrome language over the same input stream without additional information such as a separator or an end-marked two-part input format.

5.9 Limitations

The model is restricted to the alphabet {a,b}. Palindromes over digits, larger alphabets, words or arbitrary Unicode characters would require the alphabet and matching stack symbols to be extended. For a finite alphabet, the same general idea can be applied by pushing symbols from the first half and comparing them with the reverse-order suffix. However, a real text-based palindrome checker may also need to define whether uppercase and lowercase letters are treated as equal, whether spaces are ignored, and how punctuation is handled. Those normalization rules are not part of this language definition.

The PDA assumes that the input is a finite string and that it can make a nondeterministic midpoint choice. A practical deterministic program would usually use a different operational method, such as comparing characters from the two ends using indexed memory or a second data structure. Such a program may recognize the same property, but it is not the same as the standard one-stack PDA model described in this report.

The model checks exact symbol equality only. It does not interpret natural language, spelling, pronunciation, word boundaries or semantic meaning. For example, it does not know whether a phrase is meaningful; it only compares the symbols in the sequence. If the application is intended for human-language phrases, preprocessing rules for case folding, spaces and punctuation must be explicitly specified before recognition.

Nondeterminism is a theoretical recognition mechanism, not necessarily a direct implementation strategy. A simulator that explores all possible midpoint choices may require careful management of branches and stack configurations. The PDA also does not provide a user-friendly explanation of which pair of symbols caused a mismatch. A practical system would need to add diagnostics and possibly a more direct algorithm for locating mismatched positions. The formal model establishes membership in the language, but does not by itself define a user interface or error-reporting policy.

6. Comparative Analysis and Why Finite Automata Are Insufficient

6.1 Comparison of the Three Case Studies

Feature	Balanced parentheses	Arithmetic expressions	Palindromes
Alphabet / input	'(' and ')'	id, num, operators, parentheses	a and b
Main structure	Nested matching delimiters	Hierarchical operator and factor structure	Symmetry around a midpoint
CFG	$S \rightarrow (S)S \mid \varepsilon$	E/T/F precedence grammar	$S \rightarrow aSa \mid bSb \mid a \mid b \mid \varepsilon$
Stack purpose	Remember unmatched opening brackets	Store pending grammar symbols and expected terminals	Store first half for reverse comparison
PDA behaviour	Deterministic push/pop	General CFG-simulation PDA; may be nondeterministic	Nondeterministic midpoint guess
Acceptance condition	Input consumed; only Z0 remains	Input consumed; grammar stack cleared	Input consumed; only Z0 remains

6.2 Why a Finite Automaton Is Not Sufficient

A finite automaton has a fixed finite number of states and no auxiliary stack. It can recognize regular patterns, including many fixed-format strings and bounded counting tasks. However, the three case studies require memory that may grow with the input length. No fixed finite collection of states can store an arbitrary number of unmatched opening parentheses, an unbounded nested grammar context, or an arbitrarily long first half that must later be compared symbol by symbol.

Case study	Why stack memory is useful
Balanced parentheses	The PDA pushes one marker for each opening parenthesis and pops one marker for each closing parenthesis. The stack tracks nesting depth without a fixed upper bound in the mathematical model.
Arithmetic expressions	The stack stores pending grammar variables and terminals. It preserves the nested structure that must be matched while the input tokens are consumed.
Palindromes	The stack stores the first half of the string. After the midpoint is guessed, the remaining symbols are compared with the stored symbols in reverse order.

The balanced-parentheses language is a standard example of a non-regular language because it requires unbounded matching of opening and closing symbols. The palindrome language over {a,b} is also non-regular because recognizing it requires comparing arbitrarily distant corresponding positions. The arithmetic-expression language described by the CFG contains nested parentheses and recursive expression structures, which are naturally handled by a stack-based parser. A PDA therefore provides a suitable model for these context-free examples.

6.3 Key Observations

• The same stack mechanism can support different kinds of memory: counting unmatched delimiters, remembering pending grammar structure, or storing a prefix for later comparison.

• Balanced parentheses can be recognized deterministically because each symbol has a fixed push or pop action.

• Arithmetic parsing is naturally described through grammar expansion and terminal matching; the general CFG-to-PDA simulation may be nondeterministic.

• Palindrome recognition requires the PDA to guess the midpoint because the input does not explicitly mark where the first half ends.

• CFGs describe how valid strings are structured, while PDAs provide an operational model for recognizing those strings.

7. Overall Limitations and Future Scope

7.1 General Limitations

The three models are intentionally simplified so that their languages and stack operations can be explained clearly. They do not represent every feature of a real programming language or text-processing application. The balanced-parentheses model recognizes one delimiter type; the arithmetic grammar handles only a small set of binary operators and token types; and the palindrome model uses a two-symbol alphabet and exact symbol equality. In each case, the formal language is narrower than the range of inputs that a production application may need to process.

The models also separate syntax from meaning. A balanced sequence of parentheses is not necessarily a valid program. A syntactically valid arithmetic expression may contain undefined identifiers or lead to division by zero. A palindrome recognizer determines whether the sequence is symmetric, not whether it has linguistic meaning. Real systems generally combine lexical analysis, parsing, semantic validation, execution rules and user-facing error handling.

A further limitation is that a mathematical PDA abstracts away implementation resources. The stack is theoretically unbounded, but a computer has finite memory. Very deeply nested expressions or extremely long strings may require limits, timeouts or controlled error handling. The report also assumes that the input is delivered as a finite sequence and that the tokenization assumptions for arithmetic expressions are satisfied.

7.2 Possible Extensions

The balanced-parentheses model can be extended to multiple bracket types by adding separate stack symbols for each opening delimiter and requiring the corresponding closing delimiter to match the top symbol. It can also be integrated with a parser that validates the expression or statement contained inside each pair. The arithmetic-expression grammar can be extended with unary operators, exponentiation, function calls, identifiers with defined lexical rules, and additional precedence levels. A practical parser can use a transformed grammar and produce structured syntax errors.

The palindrome model can be generalized to a larger finite alphabet. A practical text-processing version could define a normalization stage that removes selected spaces or punctuation and performs case folding before testing symmetry. The chosen normalization policy must be part of the language specification; otherwise, different implementations may disagree about whether a phrase is a palindrome.

7.3 Future Scope

A future implementation exercise could translate each formal model into a small simulator that displays the current input position, state and stack contents. Such a simulator would make push and pop operations visible and help students compare valid and invalid traces. The arithmetic-expression case could be extended into a parser that constructs a parse tree and reports operator precedence. The balanced-parentheses case could be expanded to support multiple bracket types, and the palindrome case could compare the PDA model with a conventional two-pointer algorithm.

These extensions should preserve the distinction between the formal recognizer and the larger application. The PDA or CFG establishes whether a string belongs to a defined language. Additional software components are required for semantic meaning, resource management, diagnostics and interaction with users.

8. Oral Explanation / Demonstration Plan

Time	What to explain
0:00–0:40	Introduce CFG and PDA. Explain that the report studies three context-free language examples and that a PDA has a stack in addition to finite control.
0:40–2:00	Balanced parentheses: explain push on '(' and pop on ')'. Trace one valid input such as (()()) and one invalid input such as ()) .
2:00–3:20	Arithmetic expression: explain E, T and F; show how the grammar gives * and / higher precedence than + and -. Derive (a+b)*c.
3:20–4:30	Palindrome: explain pushing the first half, guessing the midpoint, and matching the remaining half. Demonstrate abba.
4:30–5:00	Compare the three models and explain why a finite automaton without a stack cannot handle unbounded nesting or arbitrary first-half matching.

Suggested short oral summary

“A context-free grammar defines the structure of valid strings, while a pushdown automaton recognizes them using a stack. In balanced parentheses, the stack stores unmatched opening brackets. In arithmetic expressions, it stores pending grammar symbols while checking the token sequence. In palindrome recognition, it stores the first half and compares it with the second half after guessing the midpoint. These examples show why stack memory is more powerful than the finite memory of a finite automaton for these context-free patterns.”

9. Conclusion

This report has presented three case studies using context-free grammars and pushdown automata. The balanced-parentheses language was described by the grammar S → (S)S | ε and recognized through deterministic stack operations. Each opening parenthesis pushes a marker, and each closing parenthesis removes one marker. The arithmetic-expression language was described using the E, T and F grammar levels, which encode the precedence of multiplication and division over addition and subtraction. A PDA can simulate this grammar by expanding non-terminals and matching terminals against the input tokens. The palindrome language over {a,b} was described by recursive productions that place matching symbols at both ends, while a nondeterministic PDA guesses the midpoint and compares the remaining symbols with the stack.

The accepted and rejected examples demonstrate how each model makes its decision. A valid string must satisfy the complete language definition, not merely reach a promising intermediate configuration. The ambiguity and determinism discussions also show that grammar structure and PDA behaviour are related but distinct: balanced parentheses have a deterministic push/pop recognizer, the general grammar-simulation PDA may be nondeterministic, and palindrome recognition requires a nondeterministic midpoint guess in the standard one-stack model.

The comparison explains why finite automata alone are insufficient for these examples. Their finite number of states cannot preserve an unbounded nesting depth or an arbitrarily long sequence that must later be compared. The stack of a PDA supplies the additional memory required by these context-free structures. Overall, the case studies demonstrate the practical importance of CFGs and PDAs in describing and recognizing nested, recursive and symmetric input patterns, while also identifying the boundaries between syntax recognition and full application validation.

10. References

[1] Task 3 – PDA and CFG Case Study Guide, Formal Language and Automata (21CSC301T), supplied course assignment guide, 9 pages. The report follows the guide's recommended case studies, required sections, worked examples, test cases and oral-explanation plan.

[2] Course: Formal Language and Automata (21CSC301T), B.Tech. Computer Science and Artificial Intelligence, Amrita Vishwa Vidyapeetham, Bengaluru Campus. Course context and notation as identified in the supplied Task 3 guide.

Note: The case-study definitions and example choices in this report are based on the supplied Task 3 guide. The report expands the explanations and organizes the PDA behaviour into a fuller case-study format; it does not claim experimental implementation or measured performance results.

Appendix – Final Submission Checklist

- Three case studies are included.
- Target alphabet and language are defined for each case study.
- CFG productions and start variables are included.
- PDA states, input symbols, stack symbols and acceptance conditions are explained.
- Accepted derivations or PDA stack traces are shown.
- Accepted and rejected examples are included with reasons.
- Ambiguity and determinism are discussed where relevant.
- Limitations and possible extensions are explained.
- The reason finite automata are insufficient is included.
- An oral explanation plan and references are provided.